UNIVERSITY of BRADFORD

Inheritance

Software Development COS7009-B

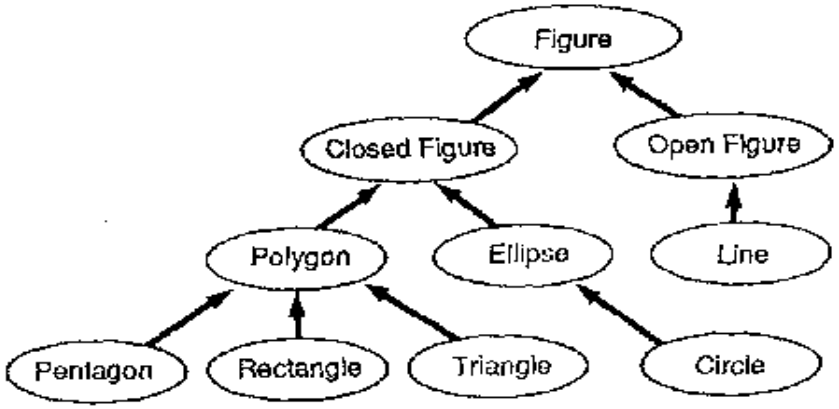
Class-Class relations

- Classes can also have a separate relationship with other classes
- the relationships forms a hierarchy
  - **hierarchy**: A body of persons or things ranked in grades, orders or classes, one above another

Is-a relationships, hierarchies

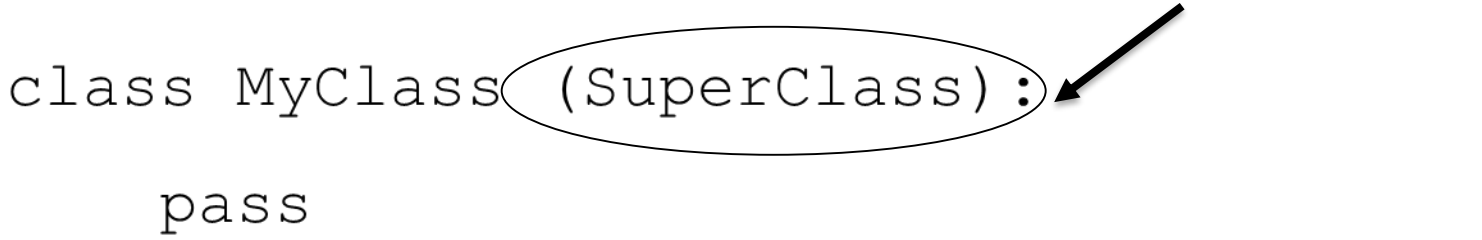
- **is-a relationship**: A hierarchical connection where one category can be treated as a specialized version of another.
  - every marketer *is an* employee
  - every legal secretary *is a* secretary

- **inheritance hierarchy**: A set of classes connected by is-a relationships that can share common code.



Class Statement

name of the class above this class in the hierarchy

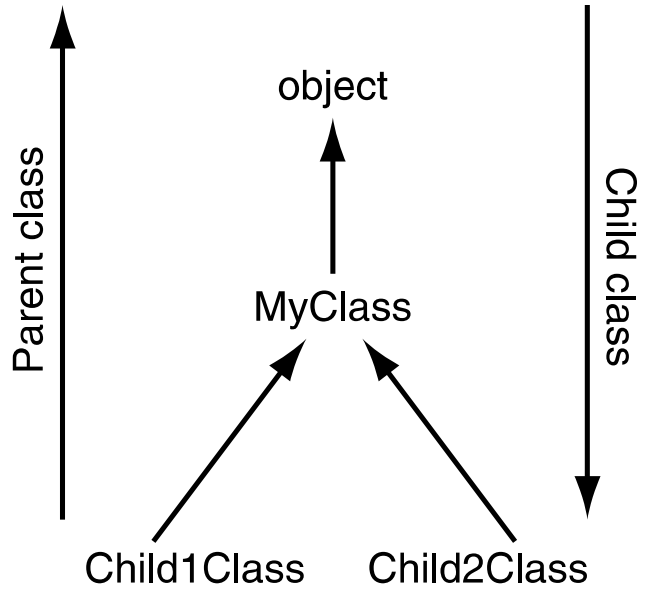


```
class MyClass (SuperClass):
    pass
```

- The top class in Python is called `object`.
- it is predefined by Python, always exists
- use `object` when you have no superclass

A Simple Class Hierarchy

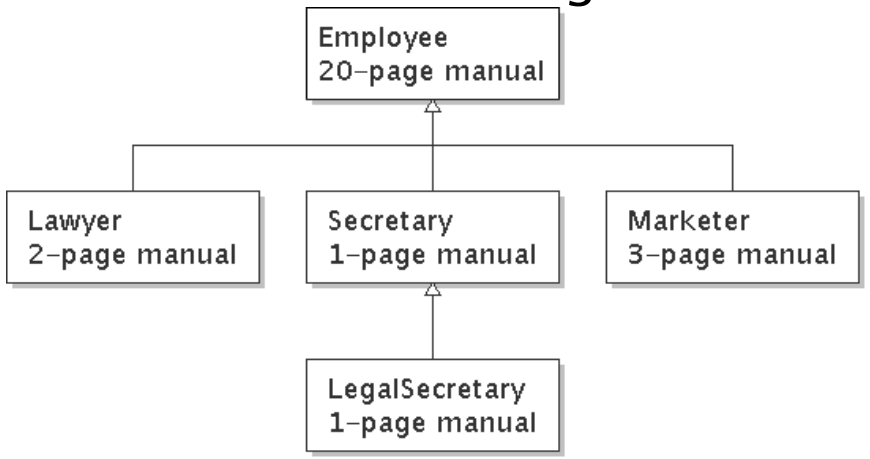
```
class MyClass (object):
    pass


class Child1Class (MyClass):
    pass


class Child2Class (MyClass):
    pass
```



Law firm employee analogy

- common rules: hours, vacation, benefits, regulations ...
  - all employees attend a common orientation to learn general company rules
  - each employee receives a 20-page manual of common rules

- each subdivision also has specific rules:
  - employee receives a smaller (1-3 page) manual of these rules
  - smaller manual adds some new rules and also changes some rules from the large manual



Separating behavior

- Why not just have a 22 page Lawyer manual, a 21-page Secretary manual, a 23-page Marketer manual, etc.?

- Some advantages of the separate manuals:
  - maintenance: Only one update if a common rule changes.
  - locality: Quick discovery of all rules specific to lawyers.

- Some key ideas from this example:
  - General rules are useful (the 20-page manual).
  - Specific rules that may override general ones are also useful.

Employee regulations

- Consider the following employee regulations:
  - Employees work 40 hours / week.
  - Employees make $40,000 per year, except legal secretaries who make $5,000 extra per year ($45,000 total), and marketers who make $10,000 extra per year ($50,000 total).
  - Employees have 2 weeks of paid vacation leave per year, except lawyers who get an extra week (a total of 3).
  - Employees should use a yellow form to apply for leave, except for lawyers who use a pink form.

- Each type of employee has some unique behavior:
  - Lawyers know how to sue.
  - Marketers know how to advertise.
  - Secretaries know how to take dictation.
  - Legal secretaries know how to prepare legal documents.

An Employee class

```
// A class to represent employees in general (20-page manual).
class Employee:
    def getHours(self):
        return 40          // works 40 hours / week

    def getSalary(self):
        return 40000.0       // $40,000.00 / year

    def getVacationDays(self):
        return 10            // 2 weeks' paid vacation

    def getVacationForm(self):
        return "yellow"      // use the yellow form
```

- Exercise: Implement class `Secretary`, based on the previous employee regulations. (Secretaries can take dictation.)

Redundant Secretary class

```
// A redundant class to represent secretaries.
class Secretary:
    def getHours(self):
        return 40            // works 40 hours / week

    def getSalary(self):
        return 40000.0     // $40,000.00 / year

    def getVacationDays(self):
        return 10            // 2 weeks' paid vacation

    def getVacationForm(self):
        return "yellow"      // use the yellow form

    def takeDictation(self,s):
        print('Taking dictation of text:', s)
```

Desire for code-sharing

- `takeDictation` is the only unique behavior in `Secretary`.

- We'd like to be able to say:

```
// A class to represent secretaries.
class Secretary:
    copy all the contents from the Employee class

    def takeDictation(self,s):
        print('Taking dictation of text:' + s)
```

Improved `Secretary` code

```
// A class to represent secretaries.
class Secretary(Employee):
    def takeDictation(self,s):
        print('Taking dictation of text:' + s)
```

- Now we only write the parts unique to each type.
  - `Secretary` inherits `getHours`, `getSalary`, `getVacationDays`, and `getVacationForm` methods from `Employee`.
  - `Secretary` adds the `takeDictation` method.

Implementing `Lawyer`

- Consider the following lawyer regulations:
  - Lawyers who get an extra week of paid vacation (a total of 3).
  - Lawyers use a pink form when applying for vacation leave.
  - Lawyers have some unique behavior: they know how to sue.

- Problem: We want lawyers to inherit *most* behavior from employee, but we want to replace parts with new behavior.

Overriding methods

- **override**: To write a new version of a method in a subclass that replaces the superclass's version.
  - No special syntax required to override a superclass method. Just write a new version of it in the subclass.

```
class Lawyer(Employee):
    // overrides getVacationForm method in Employee class
    def getVacationForm(self):
        return "pink"

    ...
```

  - Exercise: Complete the `Lawyer` class.
    - (3 weeks vacation, pink vacation form, can sue)

Lawyer class

```
// A class to represent lawyers.
class Lawyer(Employee):
    // overrides getVacationForm from Employee class
    def getVacationForm(self):
        return "pink"

    // overrides getVacationDays from Employee class
    def getVacationDays(self):
        return 15           // 3 weeks vacation

    def sue(self):
        print("I'll see you in court!")
```

- Exercise: Complete the `Marketer` class. Marketers make $10,000 extra ($50,000 total) and know how to advertise.

Marketer class

```
// A class to represent marketers.
class Marketer(Employee):
    def advertise(self):
        print("Act now while supplies last!")

    def getSalary(self):
        return 50000.0      // $50,000.00 / year
```

Levels of inheritance

- Multiple levels of inheritance in a hierarchy are allowed.
  - Example: A legal secretary is the same as a regular secretary but makes more money ($45,000) and can file legal briefs.

```
class LegalSecretary(Secretary):
    ...
```

  - Exercise: Complete the `LegalSecretary` class.

LegalSecretary class

```
// A class to represent legal secretaries.
class LegalSecretary(Secretary):
    def fileLegalBriefs(self):
        print("I could file all day!")

    def getSalary(self):
        return 45000.0      // $45,000.00 / year
```

Interacting with the superclass

Changes to common behavior

- Let's return to our previous company/employee example.

- Imagine a company-wide change affecting all employees.

  Example: Everyone is given a $10,000 raise due to inflation.
  - The base employee salary is now $50,000.
  - Legal secretaries now make $55,000.
  - Marketers now make $60,000.

- We must modify our code to reflect this policy change.

Modifying the superclass

```
// A class to represent employees (20-page manual).
class Employee:
    def getHours(self):
        return 40              // works 40 hours / week

    def getSalary(self):
        return 50000.0      // $50,000.00 / year

    ...
```

– Are we finished?

- The `Employee` subclasses are still incorrect.
  - They have overridden `getSalary` to return other values.

An unsatisfactory solution

```
class LegalSecretary(Secretary):
    def getSalary(self):
        return 55000.0
    ...

class Marketer(Employee):
    def getSalary(self):
        return 60000.0
    ...
```

- Problem: The subclasses' salaries are based on the Employee salary, but the `getSalary` code does not reflect this.

Calling overridden methods

- Subclasses can call overridden methods with `super`

  `super(subclass, self).`**method**`(`**parameters**`)`

  – Example:

```
class LegalSecretary(Secretary):
    def getSalary(self):
        baseSalary = super(LegalSecretary,self).getSalary()
        return baseSalary + 5000.0

    ...
```

  – Exercise: Modify `Lawyer` and `Marketer` to use `super`.

Improved subclasses

```
class Lawyer(Employee):
    def getVacationForm(self):
        return "pink"

    def getVacationDays(self):
        return super(Lawyer,self).getVacationDays() + 5

    def sue(self):
        print("I'll see you in court!")


class Marketer(Employee):
    def advertise(self):
        print("Act now while supplies last!")

    def getSalary(self):
        return super(Marketer,self).getSalary() + 10000.0
```

Inheritance and constructors

- Imagine that we want to give employees more vacation days the longer they've been with the company.
  - For each year worked, we'll award 2 additional vacation days.
  - When an Employee object is constructed, we'll pass in the number of years the person has been with the company.
  - This will require us to modify our `Employee` class and add some new state and behavior.
  - Exercise: Make necessary modifications to the `Employee` class.

Modified Employee class

```
class Employee:
    def __init__(self,initialYears):
        self.__years = initialYears

    def getHours(self):
        return 40

    def getSalary(self):
        return 50000.0

    def getVacationDays(self):
        return 10 + 2 * self.__years

    def getVacationForm(self):
        return "yellow"
```

Polymorphism

- **polymorphism**: Ability for the same code to be used with different types of objects and behave differently with each.
  - can print any type of object.
    - Each one displays in its own way on the console.

Polymorphism and parameters

- You can pass any subtype of a parameter's type.

```
def printInfo(empl):
        print("salary: ", empl.getSalary())
        print("v.days: ", empl.getVacationDays())
        print("v.form: ", empl.getVacationForm())
        print()

 lisa = Lawyer()
 steve = Secretary()
 printInfo(lisa)
 printInfo(steve)
```

OUTPUT:

```
salary: 50000.0          salary: 50000.0
v.days: 15               v.days: 10
v.form: pink             v.form: yellow
```

Reference

- Python Programming by Michael Dawson, ch8
- https://legacy.python.org/doc/essays/ppt/